PYTHON PROJECT FILE

STOCK PRICE ANALYSER

Track • Compare • Visualise



Python • pandas • yfinance • matplotlib

INDEX

1. Introduction 3

2. Objectives of the Project 3

3. Tools and Requirements 4

4. Important Terms Explained Simply 5

5. How the Program Works (Flowchart) 6

6. Source Code 7

7. Code Explanation (Step by Step) 8

8. Output 10

9. Understanding the Graph 11

10. Advantages, Limitations and Future Scope 11

11. Conclusion 12

1 Introduction

A **stock** is a small piece of ownership in a company. When you buy a stock of a company, you become a tiny owner of that company. The price of a stock keeps changing every day depending on demand, news and the company's performance.

People who invest in the stock market must study these price changes carefully. But looking at hundreds of numbers is boring and confusing. A **graph** makes the same data easy to understand in just a few seconds.

This project, **Stock Price Analyser**, is a Python program that does this work automatically. You only type the **symbol** of a company (like *RELIANCE.NS*) and the program will:

- fetch the company name and its current price from the internet,
- tell you how much the price has changed since yesterday,
- download old price data for any dates you choose, and
- draw a colourful graph with **7-day** and **30-day moving averages**.

In one line
It is a mini stock-market dashboard that runs in a Jupyter Notebook and uses real data from Yahoo Finance.

2 Objectives of the Project

Learn Python	Use Real Data
Practise loops, conditions, input/output, functions and exception handling in a real project.	Work with live market data from the internet instead of made-up numbers.

Data Analysis	Data Visualisation
Use pandas DataFrames to store, clean and calculate on the price data.	Turn numbers into a clear line graph using matplotlib.

Find Trends	User Friendly Tool
Use moving averages to understand whether the price is going up, down or sideways.	Make a program that anyone can use by typing just a stock symbol.

3 Tools and Requirements

Python Libraries Used

Library	Imported as	What it does in this project
pandas	pd	Stores the price data in a table (DataFrame) and calculates the moving averages.
numpy	np	Helps with number calculations. (Imported for future use.)
matplotlib	plt	Draws the line graph of the daily price and the moving averages.
seaborn	sns	Makes graphs prettier. (Imported for future use.)
yfinance	yf	Downloads company information and old price data from Yahoo Finance.

Software and Hardware

Item	Details
Language	Python 3.x
Editor	Jupyter Notebook (Anaconda)
Operating System	Windows 10 / 11 (also works on Mac and Linux)
Internet	Required, because data comes live from Yahoo Finance
Hardware	Any normal computer with at least 4 GB RAM

How to install the libraries
Open Command Prompt and type: `pip install pandas numpy matplotlib seaborn yfinance`

4 Important Terms Explained Simply

Stock Symbol (Ticker)

A short code for a company. Example: **RELIANCE.NS** is Reliance on India's NSE. Use **.NS** for NSE and **.BO** for BSE; US stocks like **AAPL** need no ending.

Current Price

The latest price at which the stock is being bought and sold right now.

Previous Close

The price at the end of the last trading day. We compare it with today's price.

Change and Change %

Change = Current Price - Previous Close.
Change % = (Change / Previous Close) x 100.

Moving Average (MA)

The average of the last few days' prices. It moves forward every day and smooths out the zig-zag ups and downs, so the real trend is easy to see.

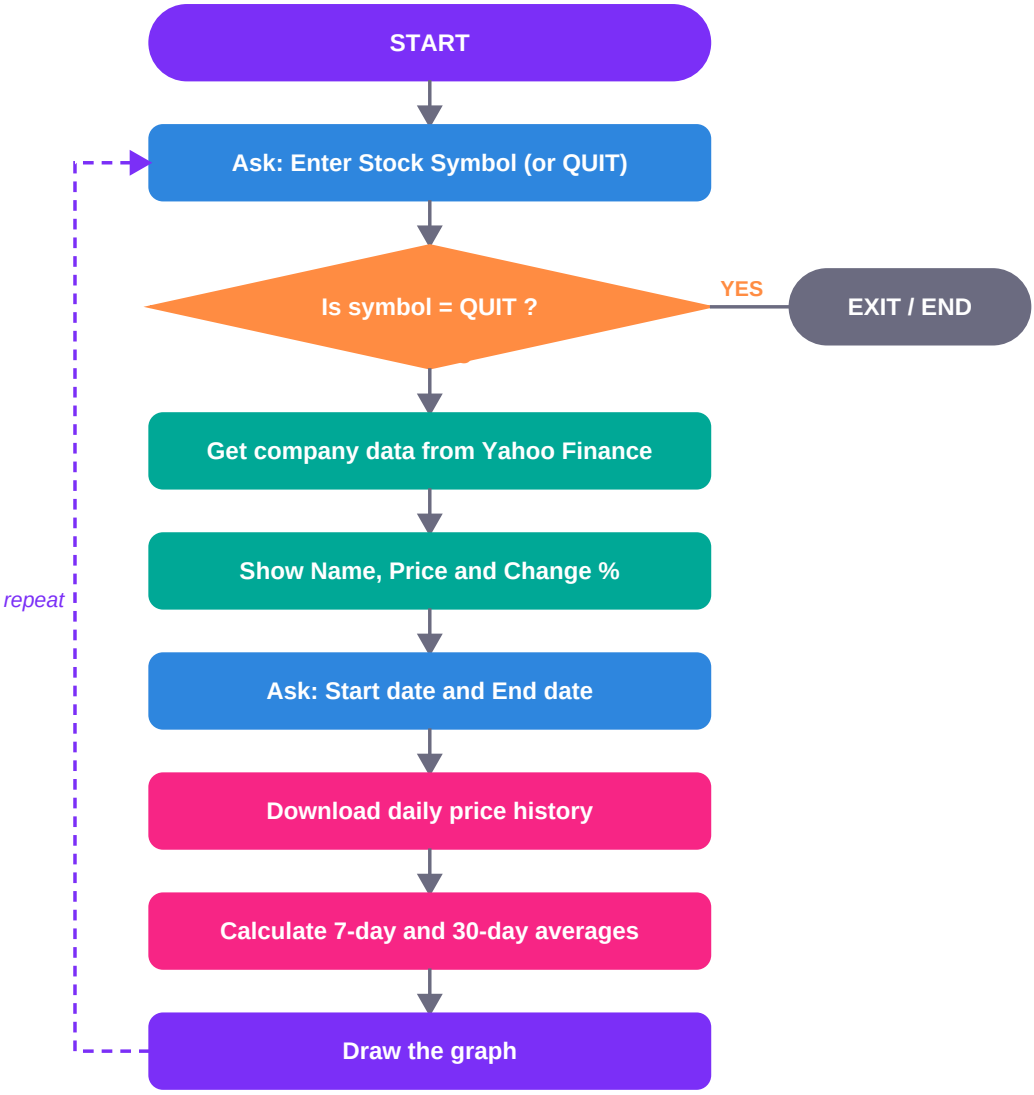
7-Day vs 30-Day MA

7-day MA reacts quickly to price changes (short-term trend). 30-day MA moves slowly (long-term trend).

Easy example of Moving Average
If a stock closed at 100, 102, 98, 101, 99, 103, 105 in the last 7 days, the 7-day MA = (100+102+98+101+99+103+105) / 7 = **101.14**. Tomorrow we drop the first number, add the new one, and find the average again.

5 How the Program Works (Flowchart)

The program runs inside a **while True** loop, so you can check as many companies as you like. It stops only when you type **QUIT**. Here is the full journey of the program:



6 Source Code

The code is written in two Jupyter cells. **Cell 1** imports the libraries and **Cell 2** contains the main program.

Cell 1: Importing Libraries

In [1]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf
```

Cell 2: Main Program

In [2] (Part A)

```
print("==========Complete Stock Analyser==========")
while True:
    symbol=input("Enter the Stock Symbol or Quit to Exit").upper()

    if symbol=="QUIT":
        print("Exiting Program. Thanks for Using it")
        break

    try:
        stock=yf.Ticker(symbol)
        info=stock.info

        name=info.get("longName","N/A")
        price=info.get("currentPrice")
        prev_close=info.get("previousClose")
        currency=info.get("currency","USD")

        print("\n ==================================")
        print(f" Company Name: {name}")
        print(f" Current Price: {price} {currency}")

        if price and prev_close:
            change=price-prev_close
            percent=(change/prev_close)*100

            print(f" Change: {change:.2f} {percent:.2f}%")

        print("\n ==================================")

```

In [2] (Part B, continued)

```
30          start_date=input("Enter the start date (yyyy-mm-dd):")
31          end_date=input("Enter the end date (yyyy-mm-dd):")
32          df = yf.download(symbol, start = start_date, end = end_date, interval="1d")
33          if isinstance(df.columns, pd.MultiIndex):
34              df.columns = df.columns.get_level_values(0)
35
36          df["7_day_MA"] = df["Close"].rolling(window=7).mean()
37          df["30_day_MA"] = df["Close"].rolling(window=30).mean()
38
39          # graphical representation
40          plt.figure(figsize=(10,6))
41          plt.plot(df["Close"],label="Daily Price")
42          plt.plot(df["7_day_MA"],label="7 Day MA")
43          plt.plot(df["30_day_MA"],label="30 Day MA")
44          plt.title(f" {name} : Stock Price Analysis")
45          plt.xlabel("Date")
46          plt.ylabel("Price")
47          plt.legend()
48          plt.grid(True)
49          plt.show()
50
51      except Exception as e:
52          print("Invalid Symbol or Network Error")
```

7 Code Explanation (Step by Step)

1 — Lines 1-2: Welcome message and loop

The program prints a heading, then **while True** starts a loop that keeps running until we stop it.

2 — Lines 3-7: Taking the symbol

input() asks for the stock symbol and **.upper()** converts it into capital letters, so *reliance.ns* and *RELIANCE.NS* both work. If the user types **QUIT**, the program prints a goodbye message and **break** ends the loop.

3 — Line 9: try block

Everything that can go wrong (wrong symbol, no internet) is kept inside **try**. This protects the program from crashing.

4 — Lines 10-16: Getting company data

yf.Ticker(symbol) connects to Yahoo Finance. **stock.info** returns a dictionary. With **.get()** we pick the company name, current price, previous close and currency. If a value is missing, .get() gives a default value instead of an error.

5 — Lines 18-20: Showing details

print() with an f-string shows the company name and the price in a neat format.

6 — Lines 22-26: Price change

If both prices are available, the program finds **change** and **percent** and prints them up to 2 decimal places using **:.2f**.

7 — Lines 30-32: Asking the dates

The user types a start date and an end date in **yyyy-mm-dd** format. **yf.download()** brings the daily (**interval="1d"**) price data into a pandas DataFrame called **df**.

8

Lines 33-34: Fixing the columns

Sometimes yfinance returns two-level column names (MultiIndex). These lines keep only the first level, so we can easily use **df["Close"]**.

9

Lines 36-37: Moving averages

.rolling(window=7).mean() takes 7 days at a time and finds their average. Same for 30 days. The results are saved as new columns.

10

Lines 39-49: Drawing the graph

plt.figure sets the size. Three **plt.plot()** lines draw the daily price and both averages. Then title, labels, legend and grid are added and **plt.show()** displays the graph.

11

Lines 51-52: except block

If anything fails inside try, this message is shown: *Invalid Symbol or Network Error*, and the loop continues.

8 Output

Sample Run: Reliance Industries (symbol: RELIANCE.NS, dates: 1 Jan 2021 to end of 2024)

Part 1: Company details shown in the console

Console Output

```
==========Complete Stock Analyser==========

 ==================================
 Company Name: Reliance Industries Limited
 Current Price: 1337.1 INR
 Change: 12.70 0.96%

 ==================================
```

Part 2: The graph

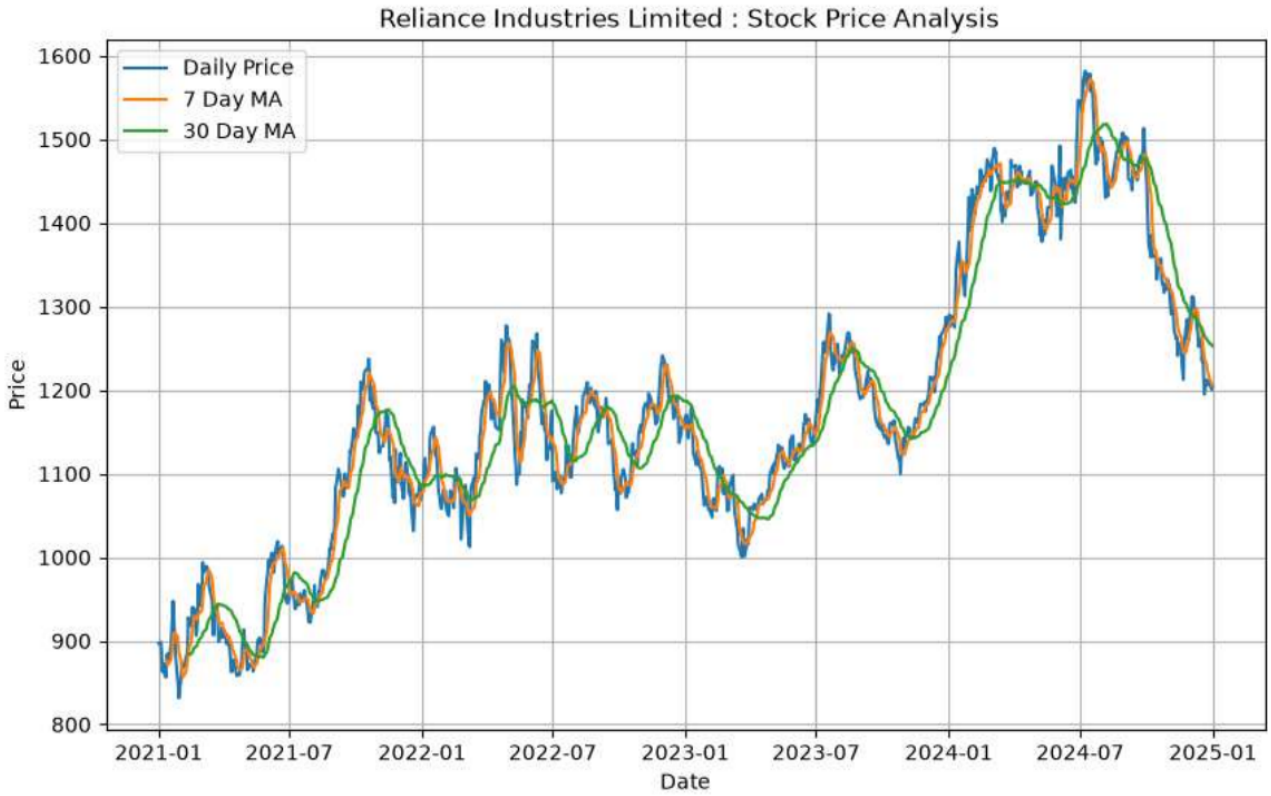


Figure: Daily closing price with 7-day and 30-day moving averages

Last step
When the user types **QUIT**, the program prints *"Exiting Program. Thanks for Using it"* and ends.

9 Understanding the Graph

Line	Colour	What it shows
Daily Price	Blue	The actual closing price of each day. It zig-zags a lot.
7 Day MA	Orange	Short-term trend. It follows the daily price closely.
30 Day MA	Green	Long-term trend. It is smoother and slower.

What can we learn from this graph? (approximate values)

- In **early 2021** the price was around **Rs 900**.
- Between 2021 and 2023 the price mostly moved **sideways** between about Rs 1000 and Rs 1250.
- From **late 2023** the price rose strongly and touched its highest point of about **Rs 1580** in **mid-2024**.
- After that, the price **fell** and ended 2024 near **Rs 1200**.
- When the orange 7-day line goes **above** the green 30-day line, it usually means the price is gaining strength. When it goes **below**, the price is losing strength.

Note
This is only a study of past data to learn Python and data analysis. It is **not** investment advice. Real investing needs much more research.

10 Advantages, Limitations and Future Scope

Advantages	Limitations
• Very easy to use • Uses real, up-to-date data • Works for any company on Yahoo Finance • Can be repeated many times without restarting • Does not crash on wrong input	• Needs internet • Data may be delayed or sometimes missing • Date format must be typed exactly right • Shows only price trends, not company news • First 6 / 29 days have no 7 / 30-day average

Future Scope (ideas to improve)

- Add **volume** bars and candlestick charts.
- Add more indicators such as **RSI** and **Bollinger Bands**.
- Compare two or more companies on the same graph.
- Save the downloaded data to a **CSV / Excel** file.
- Check the date format and show a clear message for each kind of error.
- Build a proper window or website using **Tkinter** or **Streamlit**.
- Use numpy and seaborn (already imported) for statistics such as average, highest and lowest price.

11 Conclusion

The **Stock Price Analyser** project shows how powerful Python is for real-life work. With only about fifty lines of code, we created a program that collects live stock data, calculates the daily change, finds moving averages and draws a clear graph.

While making this project I learned how to use **loops**, **conditions**, **exception handling**, **pandas DataFrames** and **matplotlib graphs**. I also understood that a good graph can explain data better than hundreds of numbers.

This project can be extended in many ways, and it has made me more interested in data analysis and finance.

Thank You!